

Experiment 'p_4_2_b_count_even_odd_seq' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_2_b_count_even_odd_seq

Natural language statement: Write a method that takes a sequence of integers and returns the number of even and odd inputs.

Method signature: p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

Ensures

- even_count == number_even(inputs)
- odd_count == number_odd(inputs)

Functional Code Given

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns the number of even and odd
→ inputs.

The signature should be:

p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

The method should respect the following contract:

ensures even_count == number_even(inputs), ensures odd_count == number_odd(inputs)

The contract uses the following dafny code:

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}
```

```
function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dme5kB14evLzNLjfgisogQkBJDFts

Model: gpt-5.5-2026-04-23

Created at: 1780486228

Finish reason: stop

Usage: 574 tokens in, and 643 tokens out

```
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
```

```

} else {
  var e, o := p_4_2_b_count_even_odd_seq(inputs[1..]);
  if inputs[0] % 2 == 0 {
    even_count := e + 1;
    odd_count := o;
  } else {
    even_count := e;
    odd_count := o + 1;
  }
}
}

```

Final Program

```

function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

method p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count:
  ↪ int)
  ensures even_count == number_even(inputs)
  ensures odd_count == number_odd(inputs)
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
  } else {
    var e, o := p_4_2_b_count_even_odd_seq(inputs[1..]);
    if inputs[0] % 2 == 0 {
      even_count := e + 1;
      odd_count := o;
    } else {
      even_count := e;
      odd_count := o + 1;
    }
  }
}
}

```

Total Token Usage

Input tokens: 574

Output tokens: 643

Reasoning tokens: 512

Sum of 'total tokens': 1217

Experiment Timings

Overall Experiment started at 1780486228144, ended at 1780486245777, lasting 17633ms (17.63 seconds)

Iteration #1 started at 1780486228145, ended at 1780486245777, lasting 17632ms (17.63 seconds)